Iterative Improvement for the Linear Ordering Problem: Neighbourhood Structures, Pivoting Rules, and Variable Neighbourhood Descent

Benjamin - Vrije Universiteit Brussel, Master in Artificial Intelligence

Heuristic Optimisation - Semester 2, Academic Year 2025-2026

Abstract

This paper evaluates iterative improvement algorithms for the Linear Ordering Problem (LOP), a combinatorial optimisation problem. Two aspects are investigated: (1) the effect of neighbourhood structure (Transpose, Exchange, Insert) and pivoting rule (first-improvement, best-improvement) under two starting-solution strategies (random and Chenery-Watanabe); and (2) the performance of Variable Neighbourhood Descent (VND) combining all three neighbourhoods in two orderings (T→E→I and T→I→E). Experiments are conducted on 78 instances. Results show that the Insert neighbourhood consistently dominates, the CW heuristic provides a superior starting solution, and best-improvement pivoting is substantially faster, yet achieves slightly worse solution quality than first-improvement. The two VND variants yield statistically indistinguishable solution quality (Wilcoxon $p = 0.39$), although T→I→E is roughly 25% faster.

1. Introduction

The Linear Ordering Problem (LOP) is a NP-hard [1] problem and asks for a permutation of n items that maximises the sum of selected entries from a given $n \times n$ cost matrix. Formally, given a matrix C, the objective is to find a permutation s that maximises $\Sigma_{i<j} c[s[i]][s[j]]$.

Because exact solvers become impractical for large instances, heuristic and iterative improvement methods are widely used. This study implements and compares a systematic set of local-search configurations and a Variable Neighbourhood Descent (VND) framework. The goals are (i) to identify the most effective neighbourhood and pivoting strategy, and (ii) to assess whether the ordering of neighbourhood structures in VND affects performance.

2. Problem Description

Let C be an n × n non-negative integer matrix. A feasible solution is a permutation $s = (s_1, \ldots, s_n)$ of {1, ..., n}. The objective value is:

$$f(s) = \Sigma_{i<j} c[s[i]][s[j]].$$

Improving s by local search requires efficiently computing the change Δ in f(s) caused by a move, rather than recomputing the full objective. Three neighbourhood structures are used, each with an O(1) or O(n) incremental delta formula:

Transpose: swap two adjacent elements s[i] and s[i+1]. Δ = c[s[i+1]][s[i]] − c[s[i]][s[i+1]]. Neighbourhood size: n-1.

Exchange: swap two arbitrary elements s[i] and s[j] (i < j). The delta accounts for the direct arc and all arcs to elements between i and j. Neighbourhood size: n(n-1)/2.

Insert: remove element at position i and reinsert at position j (j ≠ i). The delta accumulates arc-sign changes for all elements in the shifted range. Neighbourhood size: n(n-1).

3. Materials and Methods

3.1 Starting Solutions

Random (uninformed): a uniformly random permutation, seeded deterministically from the sum of all matrix entries to ensure reproducibility across runs.

Chenery-Watanabe (CW): each item i is assigned a score score[i] = $\Sigma_{i \neq j}$(c[i]c[j] - c[j]c[i]) and items are ordered by descending score. This greedy heuristic exploits pairwise dominance information and consistently yields higher initial cost values.

3.2 Pivoting Rules

First-improvement: accept the first move encountered that improves the objective, then restart the neighbourhood scan. This is faster per iteration but may accept smaller improvements.

Best-improvement: scan the entire neighbourhood, apply the best improving move, then repeat. Requires more work per iteration but exploits the steepest ascent.

3.3 Variable Neighbourhood Descent (VND)

VND systematically combines all three neighbourhoods. Starting from neighbourhood k = 1 (Transpose), it applies a first-improvement step. If an improvement is found, it restarts from k = 1; otherwise it advances to k = 2, and then k = 3. The algorithm terminates when no improvement is found in any neighbourhood. Two orderings are compared:

- **VND_TEI:** Transpose → Exchange → Insert
- **VND_TIE:** Transpose → Insert → Exchange

3.4 Instances and Statistical Analysis

Experiments are run on 78 LOP instances. Each configuration is executed once per instance. Solution quality is measured as the percentage deviation from the best-known objective value: dev(%) = 100 × (BK - f(s)) / BK, where BK is the best-known value (lower deviation is better). Statistical comparisons between algorithms use the paired two-sided Wilcoxon signed-rank test and the paired Student t-test, both at α = 0.05.

4. Results

4.1 Iterative improvement: 12 Configurations

Algorithm	Avg Dev (%)	SD Dev (%)	Total Time (s)	SD Time (s)
CW + Insert + First-imp.	1.671	0.357	699.6	7.475
CW + Insert + Best-imp.	1.844	0.331	168.1	1.691
Random + Insert + First-imp.	2.018	0.453	929.6	9.814
Random + Insert + Best-imp.	2.300	0.465	190.2	1.862
CW + Exchange + First-imp.	2.389	0.412	993.4	10.776
CW + Exchange + Best-imp.	2.958	0.442	170.8	1.742
Random + Exchange + First-imp.	2.825	0.477	1274.5	13.932
Random + Exchange + Best-imp.	3.605	0.508	191.8	1.923
CW + Transpose + First-imp.	15.839	1.414	≈0	≈0
CW + Transpose + Best-imp.	15.712	1.425	≈0	≈0
Random + Transpose + First-imp.	34.584	3.818	≈0	≈0
Random + Transpose + Best-imp.	34.446	3.822	≈0	≈0

Table 1. Average percentage deviation from best-known and total computation time for all 12 iterative improvement, configurations across 78 instance.

The results reveal a clear hierarchy among neighbourhood structures. Insert-based configurations achieve the lowest deviations (1.67-2.30%), followed by Exchange (2.39-3.61%), while Transpose alone performs very poorly (15.7-34.6%). The Transpose neighbourhood has only n-1 neighbours and is too restricted to escape local optima effectively.

The CW starting solution consistently outperforms the random start for all neighbourhood structures, confirming that a high-quality initial solution provides a better starting point for further improvement.

Regarding pivoting, best-improvement is substantially faster for both Insert and Exchange neighbourhoods, despite evaluating more moves per iteration. This suggests that best-improvement converges in significantly fewer iterations, while first-improvement requires many small improvement steps.

Solution quality remains largely comparable across configurations, although first-improvement consistently achieves slightly better results than best-improvement for both Insert and Exchange neighborhoods. In contrast, the results for Transpose are nearly identical regardless of the improvement strategy used. A possible explanation is that best-improvement, despite its more exhaustive search per iteration, can occasionally become trapped in a different local optimum of lower quality.

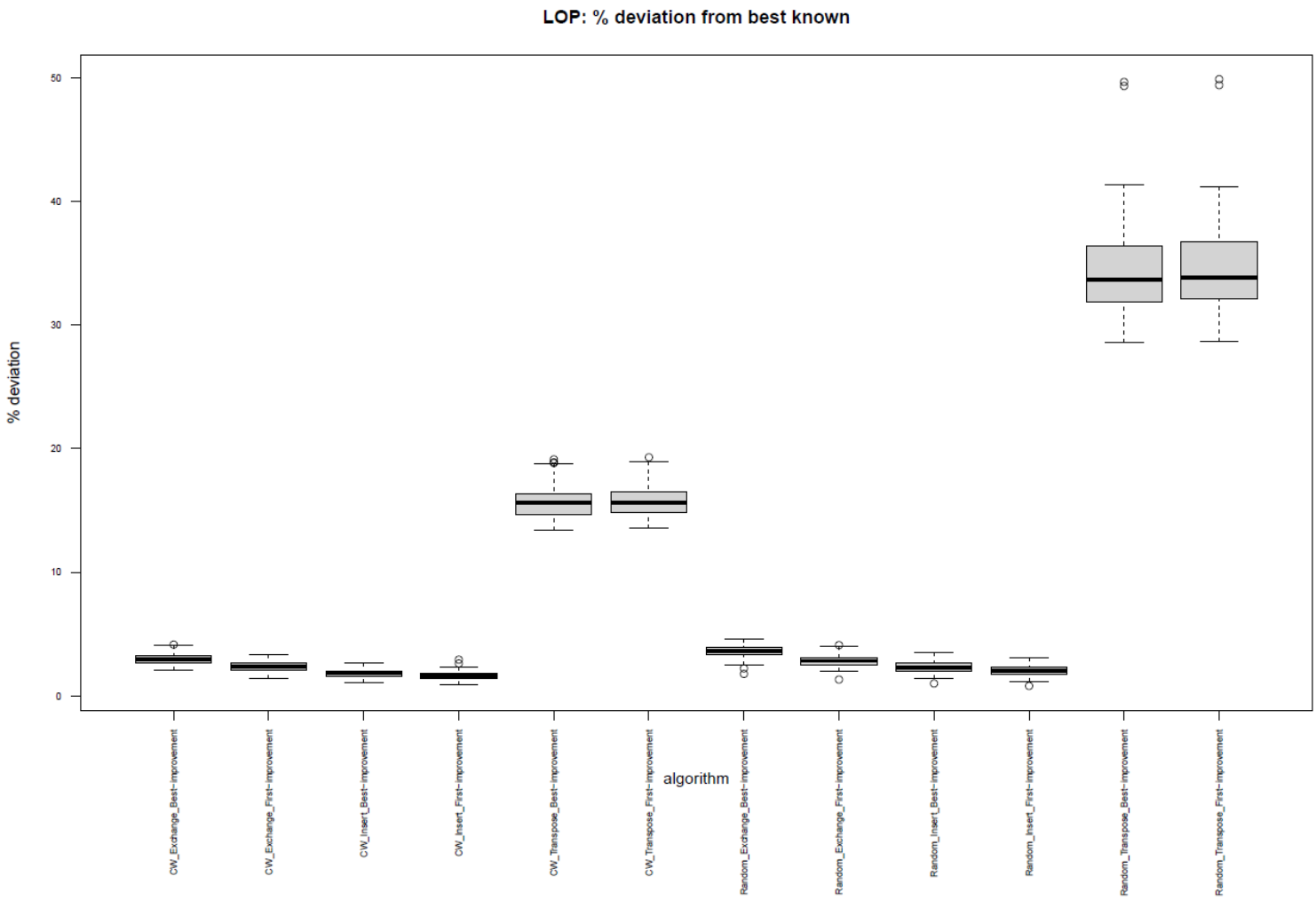

Image *1. Average percentage deviation from best-known for all 12 iterative improvement, configurations across 78 instance.*

4.2 Table 2: Student t-test p-values (paired, two-sided)

Blue = $p < 0.05$ (significant) Green = 0.05–0.20 (marginal) White = $p \geq 0.20$ Gray = diagonal

Columns 1–6

Algorithm	CW_Ex_Be	CW_Ex_Fi	CW_In_Be	CW_In_Fi	CW_Tr_Be	CW_Tr_Fi
CW_Ex_Be	1	3.92e-18	4.27e-39	2.93e-32	2.34e-74	4.49e-75
CW_Ex_Fi	3.92e-18	1	4.68e-21	2.72e-23	1.92e-77	5.96e-78
CW_In_Be	4.27e-39	4.68e-21	1	2.07e-4	3.46e-78	9.12e-79
CW_In_Fi	2.93e-32	2.72e-23	2.07e-4	1	4.67e-80	1.52e-80
CW_Tr_Be	2.34e-74	1.92e-77	3.46e-78	4.67e-80	1	9.08e-8
CW_Tr_Fi	4.49e-75	5.96e-78	9.12e-79	1.52e-80	9.08e-8	1
Ra_Ex_Be	4.57e-16	1.89e-33	3.77e-45	7.54e-45	9.92e-73	3.73e-73
Ra_Ex_Fi	1.78e-2	4.79e-12	1.49e-29	1.16e-29	2.81e-74	9.38e-75
Ra_In_Be	2.56e-15	1.73e-1	2.10e-11	4.58e-16	1.50e-75	3.59e-76
Ra_In_Fi	4.29e-22	1.10e-7	2.72e-3	6.59e-8	1.78e-76	1.56e-76
Ra_Tr_Be	5.60e-71	4.06e-72	5.61e-73	1.17e-73	4.78e-58	7.49e-58
Ra_Tr_Fi	4.06e-71	3.00e-72	3.82e-73	8.44e-74	2.68e-58	4.26e-58

Columns 7–12

Algorithm	Ra_Ex_Be	Ra_Ex_Fi	Ra_In_Be	Ra_In_Fi	Ra_Tr_Be	Ra_Tr_Fi
CW_Ex_Be	4.57e-16	1.78e-2	2.56e-15	4.29e-22	5.60e-71	4.06e-71
CW_Ex_Fi	1.89e-33	4.79e-12	1.73e-1	1.10e-7	4.06e-72	3.00e-72
CW_In_Be	3.77e-45	1.49e-29	2.10e-11	2.72e-3	5.61e-73	3.82e-73
CW_In_Fi	7.54e-45	1.16e-29	4.58e-16	6.59e-8	1.17e-73	8.44e-74
CW_Tr_Be	9.92e-73	2.81e-74	1.50e-75	1.78e-76	4.78e-58	2.68e-58
CW_Tr_Fi	3.73e-73	9.38e-75	3.59e-76	1.56e-76	7.49e-58	4.62e-58
Ra_Ex_Be	1	1.79e-21	9.80e-33	1.08e-39	4.05e-70	2.62e-70
Ra_Ex_Fi	1.79e-21	1	4.99e-11	3.84e-19	2.39e-71	1.60e-71
Ra_In_Be	9.80e-33	4.99e-11	1	3.27e-5	1.03e-71	7.68e-72
Ra_In_Fi	1.08e-39	3.84e-19	3.27e-5	1	1.41e-72	9.82e-73
Ra_Tr_Be	4.05e-70	2.39e-71	1.03e-71	1.41e-72	1	1.12e-5
Ra_Tr_Fi	2.62e-70	1.60e-71	7.68e-72	9.82e-73	1.12e-5	1

4.3 Table 3: Wilcoxon signed-rank test p-values (paired, two-sided)

Blue = $p < 0.05$ (significant) Green = 0.05–0.20 (marginal) White = $p \geq 0.20$ Gray = diagonal

Columns 1–6

Algorithm	CW_Ex_Be	CW_Ex_Fi	CW_In_Be	CW_In_Fi	CW_Tr_Be	CW_Tr_Fi
CW_Ex_Be	1	2.74e-13	2.51e-14	2.61e-14	2.51e-14	2.51e-14
CW_Ex_Fi	2.74e-13	1	3.44e-13	8.73e-14	2.51e-14	2.51e-14
CW_In_Be	2.51e-14	3.44e-13	1	1.67e-4	2.51e-14	2.51e-14
CW_In_Fi	2.61e-14	8.73e-14	1.67e-4	1	2.51e-14	2.51e-14
CW_Tr_Be	2.51e-14	2.51e-14	2.51e-14	2.51e-14	1	1.46e-7
CW_Tr_Fi	2.51e-14	2.51e-14	2.51e-14	2.51e-14	1.46e-7	1
Ra_Ex_Be	8.54e-12	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14
Ra_Ex_Fi	2.58e-2	8.06e-10	3.06e-14	3.31e-14	2.51e-14	2.51e-14
Ra_In_Be	3.37e-12	2.91e-1	9.16e-10	5.99e-12	2.51e-14	2.51e-14
Ra_In_Fi	5.93e-14	8.01e-8	1.95e-3	3.17e-7	2.51e-14	2.51e-14
Ra_Tr_Be	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14
Ra_Tr_Fi	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14

Columns 7–12: Random algorithms

Algorithm	Ra_Ex_Be	Ra_Ex_Fi	Ra_In_Be	Ra_In_Fi	Ra_Tr_Be	Ra_Tr_Fi
CW_Ex_Be	8.54e-12	2.58e-2	3.37e-12	5.93e-14	2.51e-14	2.51e-14
CW_Ex_Fi	2.51e-14	8.06e-10	2.91e-1	8.01e-8	2.51e-14	2.51e-14
CW_In_Be	2.51e-14	3.06e-14	9.16e-10	1.95e-3	2.51e-14	2.51e-14
CW_In_Fi	2.51e-14	3.31e-14	5.99e-12	3.17e-7	2.51e-14	2.51e-14
CW_Tr_Be	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14
CW_Tr_Fi	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14	2.51e-14
Ra_Ex_Be	1	2.10e-13	2.83e-14	2.51e-14	2.51e-14	2.51e-14
Ra_Ex_Fi	2.10e-13	1	2.20e-10	4.15e-13	2.51e-14	2.51e-14
Ra_In_Be	2.83e-14	2.20e-10	1	4.76e-5	2.51e-14	2.51e-14
Ra_In_Fi	2.51e-14	4.15e-13	4.76e-5	1	2.51e-14	2.51e-14
Ra_Tr_Be	2.51e-14	2.51e-14	2.51e-14	2.51e-14	1	4.16e-7
Ra_Tr_Fi	2.51e-14	2.51e-14	2.51e-14	2.51e-14	4.16e-7	1

For each comparison, **the null hypothesis (H_0) states that the mean difference in deviation percentage between the two algorithms is zero**, meaning they perform the same on average. **The alternative hypothesis (H_a) states that this mean difference is not zero,** indicating a difference in performance. Because paired tests are used, the comparison is performed instance-by-instance, rather than treating the samples as independent. **A p-value smaller than $\alpha = 0.05$ leads to rejection of H_0**, indicating a statistically significant difference, while a p-value greater than or equal to 0.05 means there is insufficient evidence to conclude a difference in performance.

Statistical tests (paired t-test and paired Wilcoxon test, $\alpha = 0.05$) show that 65 out of the 66 pairwise comparisons (diagonal not included) among the 12 configurations are statistically significant. Only one pair CW_Exchange_First-improvement vs Random_Insert_Best-improvement does not show a statistically significant difference ($p = 0.173$ for the t-test, 0.291 for the Wilcoxon test). All other comparisons are statistically significant under the chosen significance level. In other words, nearly every observed performance difference reflects a real effect rather than random variation.

4.4 VND Results

Configuration	Avg Dev (%)	SD Dev (%)	Total Time (s)	SD Time (s)
VND_TIE (T→I→E)	1.726	0.410	404.7	4.248
VND_TEI (T→E→I)	1.766	0.373	535.6	5.796

Table 4. Average percentage deviation and total computation time for the two VND variants across 78 instances.

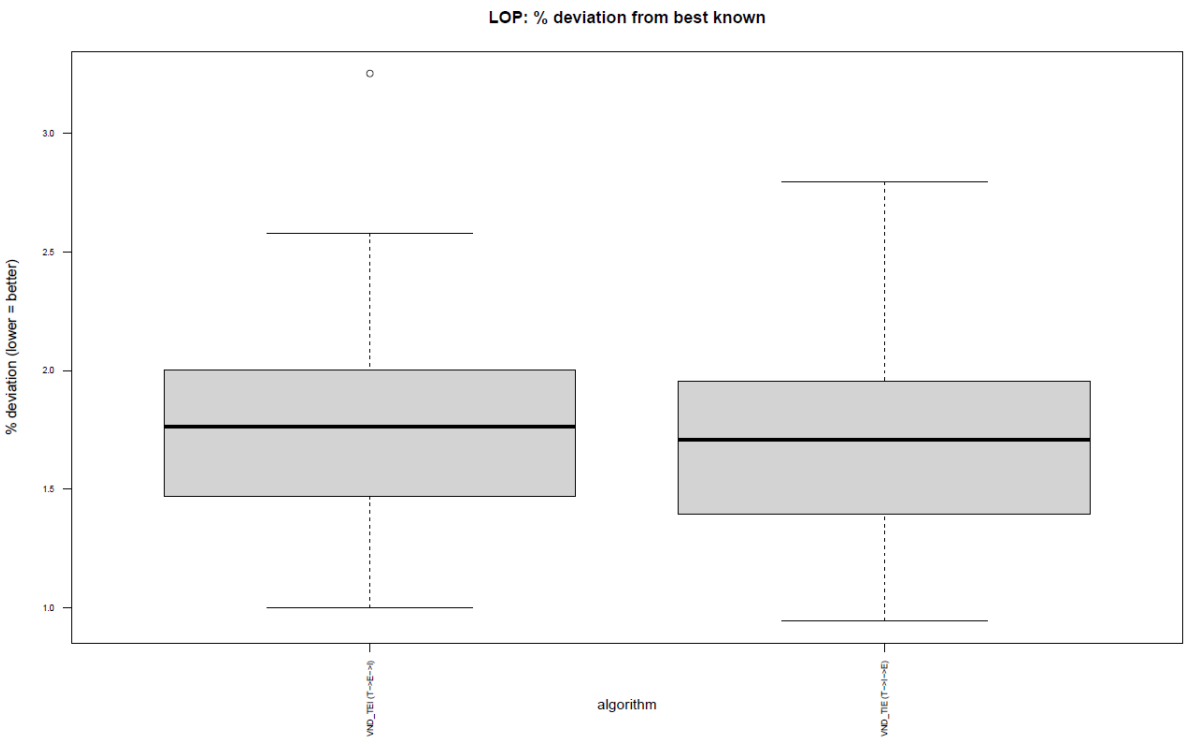

Image 2. Average percentage deviation for the two VND variants across 78 instances.

Both VND variants achieve markedly better solution quality than any single-neighbourhood iterative improvement configuration, confirming the benefit of combining multiple neighbourhood structures. VND_TIE reaches an average deviation of 1.726% and VND_TEI 1.766%, both far below the best single-neighbourhood result (CW+Insert+FI: 1.671%).

The difference between the two orderings is small: VND_TIE is approximately 25% faster (404.7 s vs 535.6 s) because placing Insert before Exchange means the stronger Insert neighbourhood finds improvements earlier, reducing the number of Exchange evaluations needed.

Test	p-value	Significant?
Paired t-test	0.4132	No
Wilcoxon signed-rank	0.3880	No

Table 5. Statistical test results comparing VND_TIE and VND_TEI (paired, two-sided, α = 0.05).

Both the paired t-test ($p = 0.413$) and the Wilcoxon signed-rank test ($p = 0.388$) fail to reject the null hypothesis at the 5% significance level, indicating no statistically significant difference in solution quality between the two VND variants. Since no significant difference in quality is observed, VND_TIE may be preferred due to its lower computation time.

5. Discussion

The experimental findings are consistent with theoretical expectations. Insert has the largest neighbourhood ($n(n-1)$ moves) and the greatest structural diversity, enabling escape from local optima that trap Transpose or Exchange. The CW heuristic captures global dominance information and places the search in a more promising region of the solution space from the outset.

The counter-intuitive result that first-improvement sometimes outperforms best-improvement in solution quality (e.g., CW+Insert: 1.671% vs 1.844%) can be explained by the fact that accepting the first improving move introduces more randomness into the search process, which can lead to exploring different regions of the solution space and occasionally reaching better local optima than always choosing the best immediate improvement.

For VND, the neighbourhood ordering matters primarily for runtime rather than quality. The cheapest neighbourhood (Transpose) is placed first in both variants as a fast filter, while Insert and Exchange swap positions. Since there is no statistically significant quality difference, the choice of ordering can be made purely based on efficiency, favouring T→I→E.

6. Conclusion

This study compared 14 iterative improvement configurations for the LOP (12 single-neighbourhood variants and 2 VND variants) across 78 search problem instances. The main conclusions are:

1. **Neighbourhood structure** is the dominant factor: Insert >> Exchange >> Transpose.
2. **Starting solution** matters: CW consistently outperforms random initialisation.
3. **Pivoting rule:** Best-improvement is faster and achieves comparable, though slightly lower, quality than first-improvement for both insert and exchange.
4. **VND** improves over single-neighbourhood search; the two orderings are statistically equivalent in quality, but T→I→E is faster and thus preferred.

Future work could investigate iterated local search or simulated annealing on top of the VND framework to further escape local optima.

References

[1] M. Grötschel, M. Jünger, and G. Reinelt, "A cutting plane algorithm for the linear ordering problem," *Operations Research*, vol. 32, no. 6, pp. 1195–1220, 1984.

[2] T. Stützle, "Local search algorithms for combinatorial problems," Ph.D. dissertation, TU Darmstadt, 1998.

[3] P. Hansen and N. Mladenović , "Variable neighbourhood search," *Computers & Operations* Research, vol. 24, no. 11, pp. 1097-1100, 1997.

[4] H. B. Chenery and T. Watanabe, "International comparisons of the structure of production," *Econometrica*, vol. 26, no. 4, pp. 487-521, 1958.